

Graphical Models

CSC 5666: Advanced Machine Learning, Fall 2026
Professor Austin P. Wright: Course Notes

Abstract

Introduction to the language of Probabilistic Graphical Models including Directed, undirected, and conditional graphical models as representations of factorization and conditional independence. Closely follows Chapter 4 of the textbook.

I Introduction

In general, a probabilistic model over the random variables $X_{1:D}$ is a joint distribution $p(x_{1:D})$. This is our fundamental object of analysis, since from a joint distribution any marginal or conditional distribution can be derived. However, in general, the parameters needed to represent a full joint distribution in its entirety is combinatorially large.

However, if we know something about the domain, and thus the conditional independence of the different variables, we can save an enormous amount of space by *factoring* the joint distribution in terms of conditional distributions. Probabilistic Graphical Models gives us a *language for specifying a conditional factorization structure of the joint distribution*.

Discussion

Consider an unstructured joint distribution for D categorical variables with K states.

1. How many parameters are required in order to represent the distribution in a table?¹
2. How many parameters does the unstructured joint need for $D = 20$ binary variables?²
3. If we knew that all D variables were *independent*, how many parameters are required in order to represent the distribution in a table?³

A **probabilistic graphical model** (PGM) uses a graph whose nodes are random variables and whose edges encode conditional-independence (CI) assumptions.⁴ The graph provides:

- a compact representation of a high-dimensional distribution
- CI statements that can be read off the graph without inspecting parameters
- local components that can be specified, learned, and reused separately
- a computational scaffold for sampling and inference

However, remember that determining the structure of a PGM is a *modeling choice* that enables real world problem solving. Since **all models are wrong, but some are useful**, we must very carefully interrogate these simplifications and weigh their utility and justification together.

Contents

I	Introduction	1
II	Directed Graphical Models: Bayesian Networks	2
III	Undirected Graphical Models: Markov Random Fields	10
IV	Conditional random fields	18
V	Comparing directed and undirected PGMs	20
VI	PGM extensions	24

Textbook: Chapter 4, pp. 143–218.

¹ Answer: $K^D - 1$

² Answer: 1,048,575

³ Answer: $D * (K - 1)$

⁴ In different ways for different kinds of PGMs. Nonetheless, in some respects a more descriptive name would be an *independence diagram*.

II Directed Graphical Models: Bayesian Networks

A **Bayesian network** (also belief network, directed PGM, or DPGM) is a directed acyclic graph (DAG) whose nodes are random variables. Each node X_i carries a conditional probability distribution (CPD) $p(x_i \mid x_{\text{pa}(i)})$ given its parents.

Textbook: §4.2, pp. 143–164

While the name Bayes Network would seem to imply this is only a Bayesian method, in fact that is not required and there are frequentist versions of the same thing. Nonetheless, since this class is Bayesian the name suffices.

Representing the Joint Distribution

Because the graph is acyclic, we can define a **topological order**: every parent precedes its children. For any such order, the chain rule of probability gives

Textbook: §4.2.1, pp. 143–144

$$p(x_{1:D}) = \prod_{i=1}^D p(x_i \mid x_{1:i-1}). \quad (1)$$

The central assumption of asserting a particular network are the independences in the joint of the **ordered Markov property** over the network which states

$$X_i \perp\!\!\!\perp X_{\text{pred}(i) \setminus \text{pa}(i)} \mid X_{\text{pa}(i)}, \quad (2)$$

That is each variable is conditionally independent of all of its predecessors, conditioned on its parents. The joint then factorizes as

$$p(x_{1:D}) = \prod_{i=1}^D p(x_i \mid x_{\text{pa}(i)}). \quad (3)$$

Root nodes have no parents, so their factors are marginal distributions.

Discussion

Why are we only worried about dependence on predecessors in the topological order and ignoring successors?

Definition: Bayesian network

A Bayesian network or Bayes Net consists of a pair (G, Θ) which includes a DAG $G = (V, E)$, and one conditional probability distribution $(p(x_i \mid x_{\text{pa}(i)}; \theta_i))$ per node. It represents the factorized joint distribution as

$$p(x_{1:D} \mid \Theta) = \prod_{i=1}^D p(x_i \mid x_{\text{pa}(i)}; \theta_i). \quad (4)$$

We use θ/Θ to represent the parametrization of the conditional distributions.

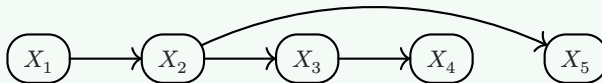
Discussion

Consider an unstructured joint distribution for D categorical variables with K states. Let us then assume the distribution is modeled using some Bayes Net where each node has at most m parents. In order to represent the joint distribution fully with conditional probability tables (CPTs), how many parameters would be required?⁵

⁵ Answer: D tables each with K^m rows and $K - 1$ free entries per row, so $D(K^m)(K - 1)$ parameters.

Discussion

Consider the following Bayes Net:



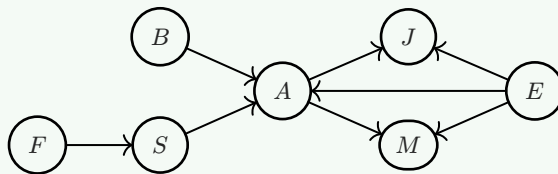
Let G be the DAG $X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4$ with a fifth node X_5 and edge $X_2 \rightarrow X_5$.

1. Using the topological order 1, 2, 3, 4, 5, write the ordered-Markov factorization of $p(x_{1:5})$, and name the CI assumption that removes each dropped conditioning variable.
2. The order 1, 2, 5, 3, 4 is also topological. Show that it yields the **same** factorization, and explain why every topological order of a DAG must.
3. Is 2, 1, 3, 4, 5 topological? If you applied the chain rule in that order and then deleted non-parents, which CI statement would you be asserting, and why is it false in general?

Discussion

Consider an example of a Bayes net with the random variables:

- B : Representing if a burglary occurs
- E : Representing if an earthquake occurs
- F : Representing if a fire occurs
- S : Representing if a smoke occurs
- A : Representing if the alarm goes off
- J : Representing if Jeff calls 911
- M : Representing if Musty calls 911



1. Write out the joint distribution using the chain rule in a topological order of the network.
2. Then simplify the distribution in terms of the conditional distributions given by the Bayes Net. Note the CI assumptions being used with each step
3. Assuming all variables are binary, what is the difference in the total number of parameters needed to represent the full joint distribution without assumptions as opposed to with the assumptions of the network.
4. Consider $A \rightarrow E$ in isolation as a two-node model. Show that the reversed model $E \rightarrow A$ can represent exactly the same set of joints. Then explain why, embedded in the full network, reversing this edge would change the implied independence model.

Gaussian Bayesian Networks

Since we frequently want to work with variables that are continuous cannot be enumerated in a CPT, we generally have to make additional assumptions about the conditional distributions in a Bayes net.

Textbook: §4.2.3, pp. 148–149

The most common CPD is a **linear Gaussian**:

$$p(x_i \mid x_{\text{pa}(i)}) = \mathcal{N}(x_i \mid \mu_i + w_i^T (x_{\text{pa}(i)} - \mu_{\text{pa}(i)}), \sigma_i^2). \quad (5)$$

Collecting the coefficients into a matrix W with $W_{ij} = 0$ whenever j is not a parent of i , and writing $S = \text{diag}(\sigma_1, \dots, \sigma_D)$, the system is

$$\mathbf{x} - \boldsymbol{\mu} = W(\mathbf{x} - \boldsymbol{\mu}) + S\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I). \quad (6)$$

In a topological order W is strictly lower triangular, so $I - W$ is invertible. Solving,

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma), \quad \Sigma = U S^2 U^T, \quad U = (I - W)^{-1}. \quad (7)$$

A missing directed edge is a **zero in the regression matrix W** . It does not in general imply a zero in the covariance Σ .

Discussion

Let $X_1 \sim \mathcal{N}(0, 1)$ and $X_2 \mid X_1 = x_1 \sim \mathcal{N}(2x_1, 3)$.

1. Write the model in the form $\mathbf{x} = W\mathbf{x} + S\boldsymbol{\epsilon}$ (assume $\boldsymbol{\mu} = \mathbf{0}$), giving W and S explicitly.
2. Compute $\mathbb{E}[\mathbf{X}]$, $\text{Var}[X_1]$, $\text{Var}[X_2]$, and $\text{Cov}(X_1, X_2)$ using $\Sigma = U S^2 U^T$.
3. Is $X_1 \perp\!\!\!\perp X_2$? Which entry of which graphical-model matrix answers the *structural* question, and why is that different from checking that $\text{Cov}(X_1, X_2) = 0$?
4. Now add $X_3 \mid X_2 = x_2 \sim \mathcal{N}(x_2, 1)$. Compute Σ and its inverse Λ . Verify that $\Lambda_{13} = 0$ while $\Sigma_{13} \neq 0$, and state the CI meaning of each.

Conditional Independence and D-Separation

D-separation (direct separation) is the process by which we can query the structure of a Bayes Net to answer questions about the conditional independence between arbitrary variables given arbitrary variables. Note that while the topology of the graph can communicate when *variables must be conditionally independent*, it does not enforce the other direction. It is possible that variables which the network considers possibly dependent may be, in reality, conditionally independent.

Textbook: §4.2.4, pp. 149–154

Common Structures (Triples)

While there is only two kinds of two variable relationships in a network (connected or disconnected), we can categorize a larger set of three node structures that will help us understand what we are looking at in a Bayes Net.



Figure 3: Causal Chain Structure

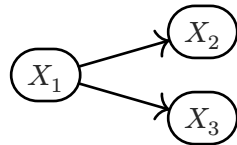


Figure 4: Common Cause Structure

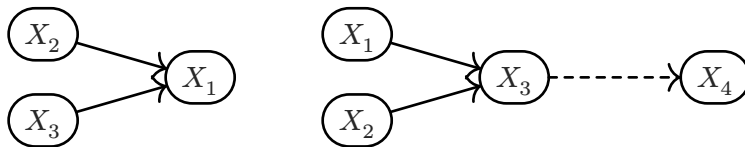


Figure 5: Common Effect Structure

⁶You can also derive these triples from the rules where a path is closed if either:

1. The arrows on the path meet either head-to-tail or tail-to-tail at a colored node
2. The arrows meet head-to-head at the node, and neither the node, nor any of its descendants is colored.

D-separation Algorithm

1. Shade all observed nodes $\{Z_1, \dots, Z_k\}$ in the graph.
2. Enumerate all undirected paths from X to Y .
3. For each path:
 1. Decompose the path into triples.
 2. If all triples are open, the path d-connects X to Y .
4. If no path d-connects X and Y , then $X \perp\!\!\!\perp Y \mid \{Z_1, \dots, Z_k\}$.

Discussion

Consider the set of open and closed triples. Notice how we show no triples containing multiple observed nodes. Explain conceptually why it must be the case that any triple containing multiple observed nodes is closed.

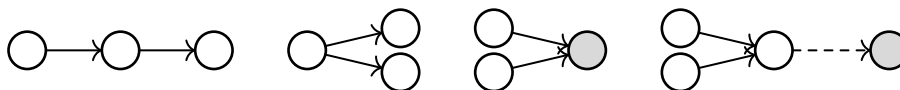


Figure 6: Open Triples

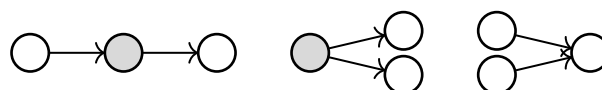
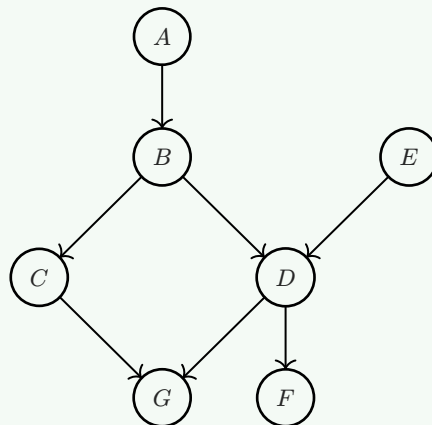


Figure 7: Closed Triples

Discussion

For the provided BayesNet, which of the following conditional independences *must* hold?

- $A \perp\!\!\!\perp F \mid D$
 - $A \perp\!\!\!\perp E$
 - $A \perp\!\!\!\perp E \mid D$
 - $A \perp\!\!\!\perp E \mid F$
 - $A \perp\!\!\!\perp E \mid D, B$
 - $A \perp\!\!\!\perp E \mid D, B$
 - $C \perp\!\!\!\perp D$
 - $C \perp\!\!\!\perp D \mid B$
 - $C \perp\!\!\!\perp D \mid B, G$
- (8)

Explaining Away and Sample Bias

In a common effect structure ($X \rightarrow Z \leftarrow Y$) with competing causes, conditioning on the common effect Z induces a *negative dependence* between the causes: evidence for one *reduces* the posterior for the other. Conditioning on a descendant of a collider, such as a selection indicator, has the same effect and produces **selection bias**. Two illustrations:

- Toss two fair coins repeatedly but record a trial only if at least one shows heads. Among recorded trials, “coin 1 is tails” forces “coin 2 is heads”: a spurious correlation created entirely by conditioning on the recording decision, a common effect.
- Gaussian DPGM $X \rightarrow Z \leftarrow Y$ with $Z = X + Y + \text{noise}$: X and Y are uncorrelated marginally, but strongly (negatively) correlated in the subpopulation $Z > c$.

Discussion

A graduate program admits applicants on research strength R and grades G , modeled as $R \rightarrow A \leftarrow G$ with $R \perp\!\!\!\perp G$ in the applicant pool.

1. Are R and G dependent in the full applicant population (That is, with no conditioning)? Explain how you can derive your answer using d-separation.
2. Among admitted students ($A = 1$), does learning that a student has weak grades change the expected research strength? Identify the active path.
3. Add a scholarship node $A \rightarrow C$ (only admitted students can receive one). A researcher analyzes a dataset containing **only scholarship recipients**. How might this bias results?

Using the properties of the Bayes net and the process of d-separation, we can also define in general the smallest set of variables to condition upon to ensure any given node is conditionally independent of all other nodes in the network, that is to *isolate* the relevant information for any individual node. We call this set the **Markov Blanket**.

Discussion

Why is the Markov blanket *not* just the parents of a node?

Definition: Markov blanket (DPGM)

The Markov blanket of node i is

$$\text{mb}(i) = \text{pa}(i) \cup \text{ch}(i) \cup \text{copa}(i), \quad (9)$$

its parents, children, and children's other parents ("co-parents"). Conditioned on its blanket, X_i is independent of all other variables, and its full conditional is⁷

$$p(x_i \mid x_{j \neq i}) \propto p(x_i \mid x_{\text{pa}(i)}) \prod_{k \in \text{ch}(i)} p(x_k \mid x_{\text{pa}(k)}). \quad (10)$$

⁷ This full conditional is used by Gibbs sampling and mean-field variational inference.

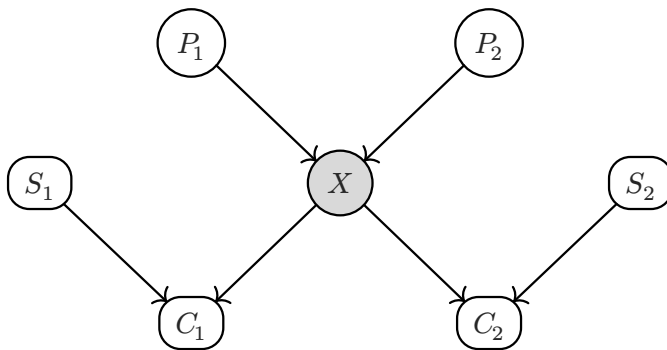


Figure 9: The directed Markov blanket of X : parents P_j , children C_j , and co-parents S_j . After textbook Figure 4.7.

Generation by Ancestral Sampling

In addition to providing information about the joint, a Bayes net gives us a generative procedure for *sampling from the joint*. Visiting the nodes in topological order and drawing

$$x_i \sim p(x_i \mid x_{\text{pa}(i)}) \quad (11)$$

produces an exact iid sample from the joint, because the product of the local sampling probabilities is exactly the represented joint. This is **ancestral sampling**. We can then use the *empirical distribution* given by repeated samples of the joint to do various forms of inference.⁸

Inference

Given query variables X_Q , evidence $X_E = e$, and hidden variables X_H , standard tasks are:

- Calculating posterior marginals $p(x_Q \mid e)$ and expectations $\mathbb{E}[g(X_Q) \mid e]$;
- Finding the marginal MAP $\arg \max_{x_Q} p(x_Q \mid e)$ and joint MAP $\arg \max_{x_Q, x_H} p(x_Q, x_H \mid e)$;
- the evidence (marginal likelihood) $p(e)$.

We can derive the posterior exactly as

$$p(x_Q \mid e) = \frac{1}{p(e)} \sum_{x_H} \prod_i p(x_i \mid x_{\text{pa}(i)}), \quad (12)$$

with integration replacing summation for continuous hidden variables. Exact inference is NP-hard in general, but for certain structures can be made to be polynomial using Variable Elimination. Otherwise variational and Monte Carlo approximations are used. These algorithms will be introduced later on.

Learning Parameters

In the analysis context where we want to learn parameters of the distributions, the structural assumptions of the network allow us to reduce the amount of data needed.⁹ For a fixed graph with parameters θ that are *global* over every data point, the MAP can be decomposed:

$$\hat{\theta} = \arg \max_{\theta} \prod_i p(\theta_i) p(\mathcal{D}_i \mid \theta_i) \quad (13)$$

This decomposition works when we observe every variable, however when there are unobserved latent variables or incomplete data we can no longer decompose over nodes, and thus must resort to optimization methods covered later.

Textbook: §4.2.5, pp. 154–155

⁸ Although we will find there will be scaling problems that will necessitate more clever sampling procedures.

Textbook: §4.2.6, p. 155

Textbook: §4.2.7, pp. 155–161

⁹ However note that this does not help us *learn the actual graph structure*, which requires a different class of learning algorithm.

Plate Notation

A frequent enough scenario in probabilistic modeling is the assumption of independent and identically distributed (IID) variables. As we showed with de-Finetti's theorem, we prefer the assumption of *exchangability* as being more useful and often more justified while covering essentially the same contexts. The assumption of exchangability is frequent enough (and helps our visual representations enough) that it has a specialized notation in PGMs called a **plate**.

A plate is just a box drawn around a repeated subgraph, labeled with its replication count or index set. Nodes outside a plate are shared, while nodes inside have one instance per index (and since the index number has no effect on the CPD, this is equivalent to exchangability).

$$p(\theta, x_{1:N}) = p(\theta) \prod_{n=1}^N p(x_n | \theta), \quad (14)$$

A plate is syntactic sugar that asserts repetition **and parameter sharing**. While it is the most frequent notation for repetition, note that not all abstractions over repetitive models can be represented with just plates (e.g. an HMM).

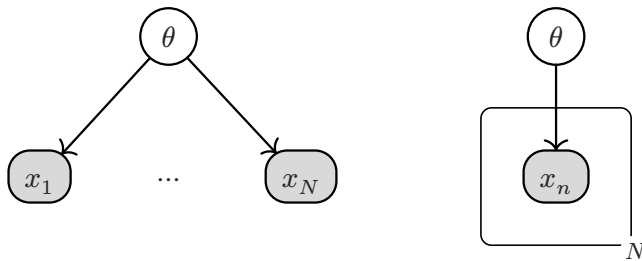


Figure 10: Left: N explicit iid observations sharing a parameter θ . Right: the same model in plate notation. After textbook Figure 4.11.

Discussion

Draw plate diagrams, for the following models:

1. Bayesian linear regression with observed inputs x_n and observed targets y_n , with global parameters of and linear gaussian CPDs.
2. Factor analysis of observed data x_n , a vector of unobserved latent factors z_n , and global parameters of and linear gaussian CPDs.
3. How are the two models different?

Discussion

Consider the plate notation for a Naive Bayes classifier shown above. Attempt to draw what the network would look like without plate notation.

Textbook: §4.2.8, pp. 161–164

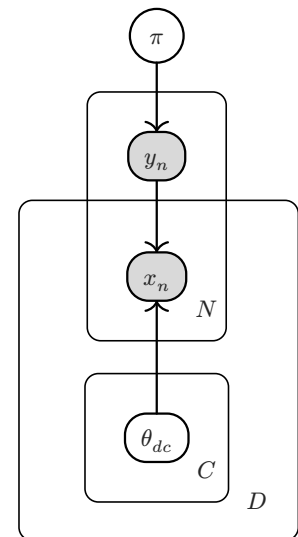


Figure 11: Naive Bayes classifier as a DPGM augmented with plate notation. After textbook Figure 4.13.

III Undirected Graphical Models: Markov Random Fields

A **Markov random field** (MRF), **Markov network**, or **undirected PGM** (UPGM) uses an undirected graph. It is natural when dependence is symmetric (spatial or relational data) or when no convenient generative order exists. Forcing directions onto such a domain, for example a 2d image lattice written as a DAG, yields awkward CI properties. Relative to DPGMs, UPGMs are more symmetric and, in discriminative form, often more accurate, but their parameters are less modular and more expensive to fit.

Textbook: §4.3, pp. 164–186

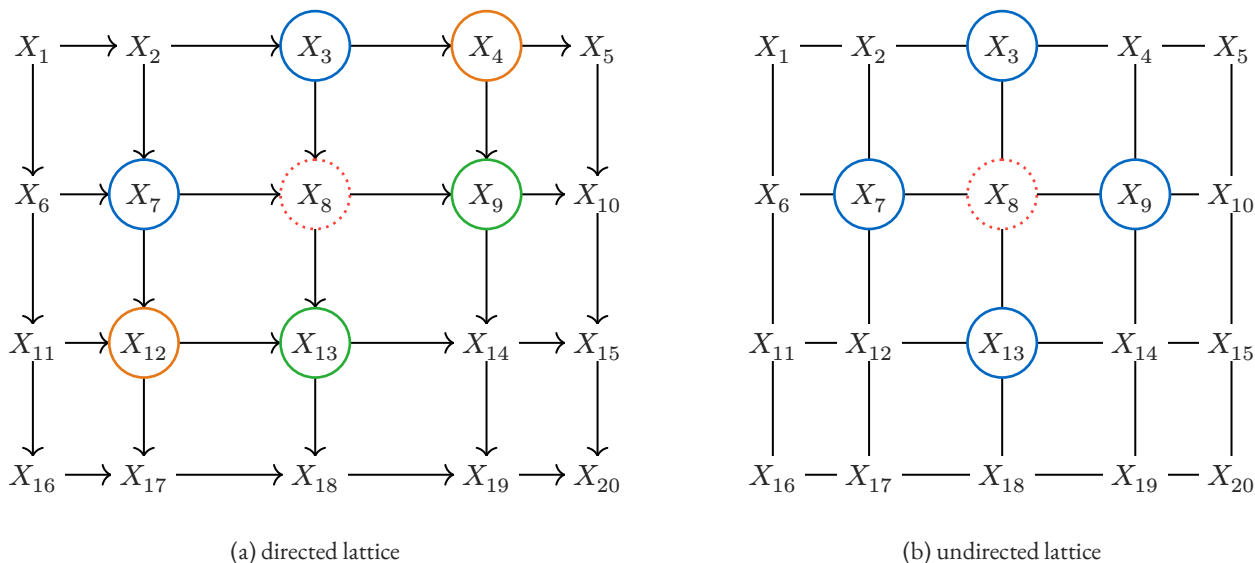


Figure 12: Left: a 2d lattice represented as a DAG, where the Markov blanket of X_8 includes its parents (blue), children (green), and co-parents (orange). Right: the same lattice as an undirected PGM, where the Markov blanket is just the four neighboring nodes (blue). After textbook Figure 4.15.

Representing the Joint Distribution

With no topological order there is no chain rule to invoke to allow application of the CI assumptions of the network. As long as the joint is *strictly positive*¹⁰, we can apply the Hammersley–Clifford theorem to write the joint, which states that we can associate a nonnegative **potential** (factor) $\psi_{c(x_c; \theta_c)}$ with each maximal clique¹¹ c in a set $\mathcal{C}$, usually the maximal cliques. The joint is¹²

$$p(x \mid \theta) = \frac{1}{Z(\theta)} \prod_{c \in \mathcal{C}} \psi_c(x_c; \theta_c), \quad Z(\theta) = \sum_x \prod_{c \in \mathcal{C}} \psi_c(x_c; \theta_c), \quad (15)$$

We can also write the distribution as

$$p(x \mid \theta) = \frac{1}{Z(\theta)} \exp(-\mathcal{E}(x; \theta)) \quad (16)$$

Textbook: §4.3.1, pp. 165–166

¹⁰ Since hard zeros, which are just *deterministic* constraints, can create independences not visible in the graph.

¹¹ A clique is a set of nodes that are all neighbors of each other. A maximal clique is a clique which cannot be made any larger without losing the clique property

¹² We call Z the **partition function**. The letter Z is used from the German word *Zustandssumme*, meaning “sum over states”. This reflects the fact that a lot of pioneering working on MRFs was done by German (and Austrian) physicists, such as Boltzmann.

Where $\mathcal{E} > 0$ is called the **energy** of the state x , defined over cliques c as

$$\mathcal{E}(x; \theta) = \sum_c \mathcal{E}(x_c). \quad (17)$$

We can see that *low energy* is associated with *high probability*. This kind of distribution is called the **Gibbs distribution**, and this kind of model is called an **energy-based model**. This kind of model is very common in some domains such as physics and biochemistry, as well as in machine learning to define certain kinds of generative models which we will cover later in the course.

Simple Example

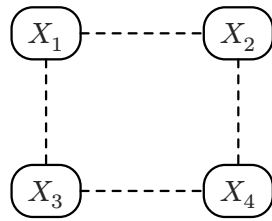


Figure 13: A pairwise MRF on a 4-cycle. Its maximal cliques are the four edges.

For this graph one valid pairwise factorization is

$$p(x_{1:4}) = \frac{1}{Z} \psi_{12}(x_1, x_2) \psi_{13}(x_1, x_3) \psi_{24}(x_2, x_4) \psi_{34}(x_3, x_4). \quad (18)$$

Discussion

Consider a strictly positive $p(x_1, x_2, x_3)$ that satisfies exactly $X_1 \perp\!\!\!\perp X_3 \mid X_2$ and no other CI statement.

1. Draw the minimal undirected graph. What are its maximal cliques?
2. By Hammersley–Clifford, write the most general factorization p can take. How many free parameters if each variable is binary, and how does this compare with the unstructured joint?
3. Come up with a **non-positive** joint on three binary variables that obeys the graph separation statement $X_1 \perp\!\!\!\perp X_3 \mid X_2$ but does **not** factorize as $\psi_{12}\psi_{23}/Z$. (Hint: *use a deterministic constraint*.)

Ising Models

Textbook: §4.3.2, pp. 166–172

In this section, we discuss some MRFs for 2d grids, that are used in statistical physics and computer vision. We then discuss extensions to other graph structures, which are useful for biological modeling and pattern completion.

Consider the 2d lattice introduced earlier. We can represent the joint distributions follows:

$$p(x) = \frac{1}{Z} \prod_{i \sim j} \psi_{ij}(x_i, x_j; \theta) \quad (19)$$

Where $i \sim j$ represents i and j being neighbors in the graph. This is a 2d lattice model. An **Ising model** is a spacial case of a 2d lattice model where the variables x_i are binary. This is frequently used in material science to represent atoms with spin as $x_i \in \{-1, +1\}$. In some systems nearby atoms prefer to have matching spins, while in other systems nearby atoms prefer to have opposite spins. We can capture this with:

$$\psi_{ij}(x_i, x_j; \theta) = \begin{cases} e^{J_{ij}} & \text{if } x_i = x_j \\ e^{-J_{ij}} & \text{if } x_i \neq x_j \end{cases} \quad (20)$$

Where J_{ij} is the *coupling strength* between nodes i and j . For all nodes not connected in the graph we set $J_{ij} = 0$. Since our model is undirected, we assume the matrix is symmetric and $J_{ij} = J_{ji}$. Frequently we simplify by assigning all edges the same strength $J_{ij} = J$ for each nonzero edge. This reduces to

$$\psi_{ij}(x_i, x_j; \theta) = \begin{cases} e^J & \text{if } x_i = x_j \\ e^{-J} & \text{if } x_i \neq x_j \end{cases} \quad (21)$$

Most frequently an Ising model is defined as an energy based model where

$$\begin{aligned} p(x) &= \frac{1}{Z(J)} \exp(-\mathcal{E}(x; J)) \\ \mathcal{E}(x) &= -J \sum_{i \sim j} x_i x_j \end{aligned} \quad (22)$$

where $\mathcal{E}(x; J)$ is the energy, and where we exploited the fact that $x_i x_j = -1$ if $x_i \neq x_j$, and $x_i x_j = +1$ if $x_i = x_j$. The magnitude of J controls the degree of coupling strength between neighboring sites. We can scale the coupling coefficient J by a temperature term T to get $J' = \frac{J}{T}$, so colder means more tightly coupled (larger J), and hotter means less tightly coupled (smaller J).

If all of the weights are negative, $J < 0$, then the spins want to be different from their neighbors. This is called an *antiferromagnetic* system, and results in a frustrated system, since it is not possible for all neighbors to be different from each other in a 2d lattice (resulting in a checkerboard on an infinite lattice).

If all the edge weights are positive, $J > 0$, then neighboring spins are likely to be in the same state, since if $x_i = x_j$, the energy term gets a contribution of $-J < 0$, and lower energy corresponds to higher probability. In the machine learning literature, this is called an associative Markov network. In the physics literature, this is called a *ferromagnetic* model.

If $J = 1$, the corresponding probability distribution will have two modes, corresponding to the all $+1$ state and the all -1 state. These are called the ground states of the system.

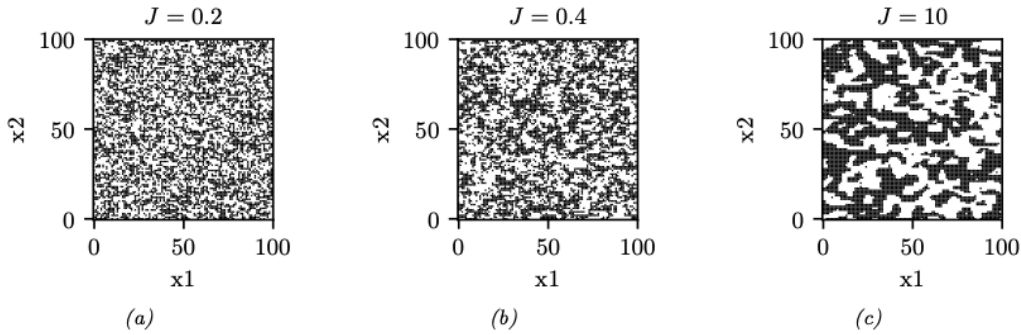


Figure 14: Samples from an associative Ising model with varying $J > 0$. From textbook figure 4.17

This model can be extended in a wide range of ways, such as adding *unary terms* in addition to pairwise terms, which in physics would be called an external field, or by extending the variables from binary to larger dimensional categorical variables, which is called a *Potts model*, which is also used in a huge range of scientific applications such as protein structure prediction.

Hopfield Networks

One particularly special kind of Ising model is the *Hopfield Network*. A Hopfield network is a *fully connected* Ising model with symmetric weights $W = W^T$. Its energy function has the form (where $x \in \{-1, +1\}$)

$$\mathcal{E}(x) = -\frac{1}{2}x^T W x \quad (23)$$

Hopfield networks are mainly used for the problem of *associative memory*. The problem goes as follows: suppose we train on a set of fully observed bit vectors, corresponding to patterns we want to memorize. Then, at test time, we present a *partial* pattern to the network. We would like to estimate the missing variables; this is called *pattern completion*, by calculating

$$\underset{x}{\operatorname{argmin}} \mathcal{E}(x) \quad (24)$$

We can solve this optimization problem using *iterative conditional modes* (ICM), in which we variable to its most likely state given its neighbors, which amounts to applying the iterative update:

$$x_{t+1} = \operatorname{sign}(W x_t) \quad (25)$$

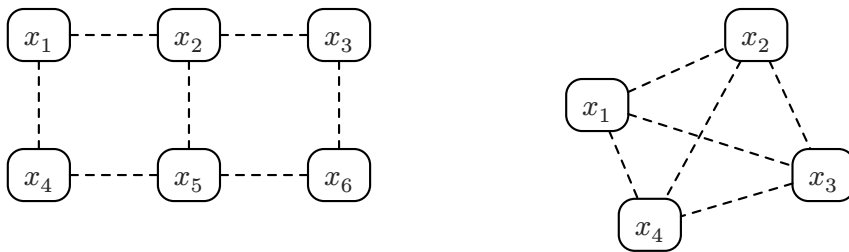


Figure 15: Left: an Ising / Potts model on a 2d lattice (pairwise edges only). Right: a Hopfield network is the fully connected case.

Boltzmann Machines

While MRFs where every variable is visible are useful, in this class we are generally more interested in learning *latent representations* that can represent high dimensional joint distributions in discrete space. One kind of such model is a **Boltzmann machine**, which is an Ising-type energy model with states in $\{0, 1\}$ as opposed to $\{-1, +1\}$, an arbitrary graph as opposed to a lattice, and the nodes partitioned into visible $\vec{v}$ and hidden $\vec{h}$.

However, in this general case inference is intractable, and even approximate inference methods can be very non-performant. Because of this we generally add in the restriction to the graph structure that it must be *bipartite between visible and hidden nodes*, that is there are no internal edges between either hidden or visible nodes, only connected from hidden to visible. This, appropriately, is called a **restricted Boltzmann machine (RBM)**¹³. This structure makes approximate inference efficient by making the hidden nodes all conditionally independent given the visible nodes¹⁴ (and vice versa). That is we can factorize as such

$$p(\vec{z}|\vec{x}) = \prod_k p(z_k | \vec{x}) \quad \text{and} \quad p(\vec{x}|\vec{z}) = \prod_d p(x_d | \vec{z}) \quad (26)$$

Typically, the hidden and visible nodes in an RBM are binary¹⁵, so the energy terms have the form $w_{dk}x_dz_k$. If $z_k = 1$, then the k^{th} hidden unit adds a term of the form $w_k^\top x$ to the energy. This can be thought of as a “soft constraint”. If $z_k = 0$, the hidden unit is not active. By turning on different combinations of constraints, we can create complex distributions on the visible data. This is an example of a *product of experts* since we have a factorization of the form

$$p(\vec{x}|\vec{z}) \propto \prod_{\{k \mid z_k=1\}} \exp(w_k^\top \vec{x}) \quad (27)$$

Which can be thought of as a very large mixture model with an *exponential* number of hidden components, corresponding to the 2^H settings of $\vec{z}$. That is $\vec{z}$ is a *distributed representation*, where a standard mixture model uses a *local representation* of $z \in \{1, \dots, K\}$, and each value corresponds to a complete prototype or exemplar of $\vec{x}$, $\vec{w}_k$, with a model of the form

$$p(\vec{x}|z = k) \propto \exp(w_k^\top \vec{x}) \quad (28)$$

Textbook: §4.3.3, pp. 172–174

¹³ Also sometimes called a **harmonium**

¹⁴ Note this is in contrast to a directed two-layer models, where the explaining away effect causes the latent variables to become “entangled” in the posterior even if they are independent in the prior.

¹⁵ However variations of RBMs for Gaussian or Categorical variables in either the hidden or visible layers have been defined as well

Discussion

Notice how the models of a *local representation* and a *distributed representation* look very similar. Explain, in plain language, what is different between the two kinds of model, and which parts of the equations for the generative distributions correspond to those differences.

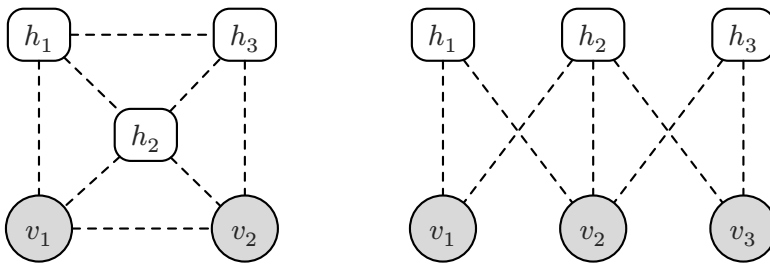


Figure 16: Left: Example of a general Boltzmann machine with arbitrary structure. Right: Example of a restricted Boltzmann machine with a bipartite structure and no intra-layer edges. After textbook Figure 4.20.

Deep Belief Nets

While the layer restriction is what allows RBMs to be tractable, we can slightly loosen the restriction and maintain many of the properties by building networks with non-intra-connecting layers, but instead of only one hidden layer, we can “stack” multiple layers. We call this a **deep Boltzmann machine**. For instance, a two layer model would have the basic form

$$p(x, z_1, z_2 \mid \theta) = \frac{1}{Z(W_1 W_2)} \exp(x^\top W_1 z_1 + z_1^\top W_2 z_2) \quad (29)$$

One important use of a deep Boltzmann machines are in **deep belief networks (DBN)**¹⁶, which use a deep RBM as a prior over a latent distributed code, and then use a directed model “decoder” to convert this to the observed data, giving the joint the form (for a two layer example as above)

$$p(x, z_1, z_2 \mid \theta) = p(x|z_1, W_1) \frac{1}{Z(W_2)} \exp(z_1^\top W_2 z_2) \quad (30)$$

DBNs are historically important models, since they can be trained fairly easily and support efficient inference, and were one of the first deep learning models to be successfully trained. They no longer as frequently used due to developments improving ways to train fully supervised neural networks on the one hand, and more relevantly for us more efficient ways to train deep Bayes Nets such as the VAE.

¹⁶This is another example of a confusing name, since a belief network is a *different thing* and a DBN is not actually a belief network. Thus sometimes you may see these called Deep Boltzmann Networks to clarify but keep the acronym.

Conditional Independence in MRFs

Just as we had to determine the CI assumptions in a Bayes Net in order to construct an appropriate model, similarly we need to understand how MRFs encode CI assumptions. Luckily this algorithm is somewhat simpler than d-separation, defined

Definition: Global Markov Property for MRFs

Given three sets of nodes A, B, C in an undirected graphical model G .

We say that $X_A \perp\!\!\!\perp X_B \mid X_C$ iff C separates A from B in the graph G .¹⁷

Discussion

Using the global Markov property, what is the *Markov Blanket* of a node in a MRF. If you are having trouble, draw some small example graphs and try to apply the property. This result is called the **undirected local Markov property**.

From the local Markov property, we can easily show that two nodes are conditionally independent given the rest of the network iff there is no direct edge between them. This is called the **pairwise Markov property**. While it is easy to show how the global property implies the local property, and how the local property implies the pairwise property, it is less clear but true that the pairwise property in turn implies the global property¹⁸. *This means that all three properties are in fact exactly equivalent!* This helps us since the pairwise property is generally much easier to reason about and empirically assess, and so we can use these properties to construct a model on which the wider class of global attributes are ensured to hold.

Since CI seems to be much easier to work with in undirected models, an alternative to d-separation is to convert a DPGM to an equivalent (with respect to CI) UPGM, and query independence on the UPGM. The conversion process is called **moralization**, and requires us to add edges between all “unmarried parents”, that is all parents of a node must be connected, after which we can remove all directions and produce the new undirected graph. Unfortunately, some information is still lost by moralization. While there are no independences implied in the moralized network that are not present in the directed network, the converse is not true as there are some conditional independences present in the directed graph that are not present in the moralized graph.

If we want to be more precise we have to construct different undirected graphs depending on the CI query $A \perp\!\!\!\perp B \mid C$. To do this we have to construct the *ancestral graph* of G with respect to $U = A \cup B \cup C$, which removes all nodes from G that are not either in U or are not ancestors of U . We can then moralize the ancestral graph and use the resulting undirected graph to query CI properties which will be equivalent between the original directed graph and the undirected moralized ancestral graph.

Textbook: §4.3.6, pp. 179–181

¹⁷ One interesting property this gives is that, unlike in a directed model, observing a node in an undirected model is *never* path-opening: undirected CI is *monotonic* ($A \perp\!\!\!\perp B \mid C$ implies $A \perp\!\!\!\perp B \mid (C \cup D)$).

¹⁸ With the only provision of requiring strict positive distributions, $p(x) > 0$, which is not an onerous or unusual requirement.

Discussion

Why can we not just simply remove the arrows in a DPGM to convert to an undirected model? Construct a simple example where doing so results in incompatible CI properties.

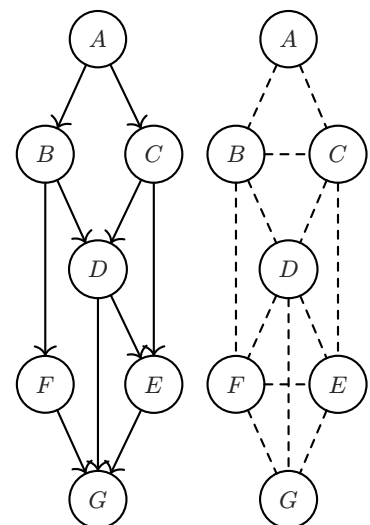


Figure 17: Directed PGM and its moralized representation. After textbook Figure 4.23.

Sampling, Inference, and Learning

Sampling and Inference

Because UPGMs have no topological order, sampling directly and performing inference can be more difficult and slow. Additionally, we cannot calculate the probability of any configuration without the value of Z . Therefore approximation methods such as MCMC or mean field variational inference are used.

Learning Parameters

Since computing the likelihood of an MRF is itself very difficult due to the need to deal with the partition function Z , when adding the difficulty in computing the full posterior over parameters, $p(\theta|\mathcal{D})$, learning in MRFs is sometimes called **doubly intractable**. As a result we generally limit analysis to point estimation methods such as MLE and MAP optimized using gradient methods.

This presents the primary computational cost in learning MRFs, computing the derivative of the (log) partition function which is used to compute the derivative of the log likelihood. We can see why this is difficult ¹⁹

$$\begin{aligned}
 \nabla_{\theta} \log(Z(\theta)) &= \frac{\nabla_{\theta} Z(\theta)}{Z(\theta)} \\
 &= \frac{1}{Z(\theta)} \nabla_{\theta} \int p(x; \theta) dx \\
 &= \frac{1}{Z(\theta)} \int \nabla_{\theta} p(x; \theta) dx \\
 &= \frac{1}{Z(\theta)} \int p(x; \theta) \nabla_{\theta} \log(p(x; \theta)) dx \\
 &= \int \frac{p(x; \theta)}{Z(\theta)} \nabla_{\theta} \log(p(x; \theta)) dx \\
 &= \mathbb{E}_{x \sim p(x; \theta)} [\nabla_{\theta} \log(p(x; \theta))]
 \end{aligned} \tag{31}$$

All this is to show that *we need to draw samples from the model at each gradient calculation / training step!* Since sampling from the model is itself hard for MRFs, often needing its own approximation or optimization process, this poses a substantial issue. Because of this issue there have been devised various efficient sampling methods, as well as alternative estimators that do not use the principle of maximum likelihood.

Maximum Pseudolikelihood Estimation

When fitting MRFs, one common alternative to maximizing likelihood is to maximize *pseudolikelihood*. Which is to optimize the product of the full conditionals. ²⁰

$$\ell_{PL}(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{d=1}^D \log(p(x_{nd} \mid \{x_{nd'} \mid d' \neq d\}, \theta)) \tag{32}$$

Textbook: §§4.3.7–4.3.9, pp. 181–186

¹⁹This utilizes the *log-derivative trick* based on the fact that

$$\nabla_{\theta} \log(p(x; \theta)) = \frac{1}{p(x; \theta)} \nabla_{\theta} p(x; \theta)$$

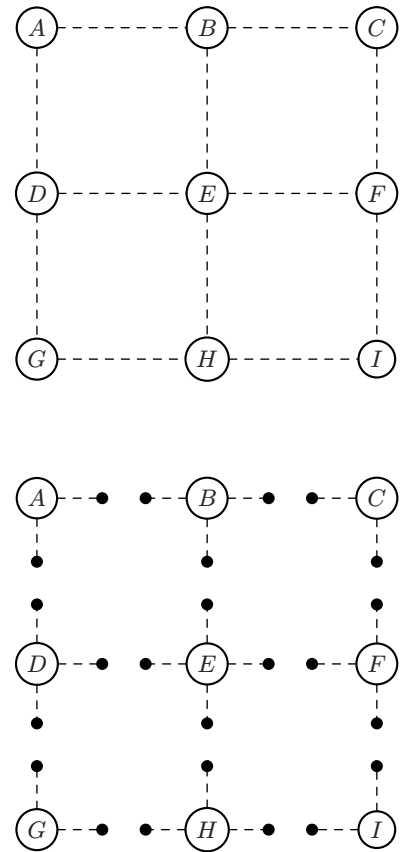


Figure 18: Example MRF (above) and Implicit representation used by pseudolikelihood (below). After textbook Figure 4.28 .

²⁰ Pseudolikelihood can be much faster than full MLE, and is equivalent for Gaussian MRFs (although this is not true in general). Empirically, pseudolikelihood has been shown to work well for Ising models, but performs worse than stochastic approximations on RBMs.

IV Conditional random fields

A **conditional random field** (CRF) is model over structured outputs y conditioned on inputs x :²¹

$$p(y | x, \theta) = \frac{1}{Z(x, \theta)} \prod_c \psi_c(y_c; x, \theta). \quad (33)$$

CRFs are useful because they capture dependencies between outputs, which allows us to perform *structured prediction*, where the output $y \in \mathcal{Y}$ we want to predict given an input x exists in a structured space with constraints on valid values of y . For example in NLP if we want to parse the grammar of a sentence we require the labels adhere to the rules of the grammar. Additionally constraints can be “soft” such as for image segmentation where we might want to prefer pixels to be classified similarly to their neighbors unless we have a sufficient signal to identify a boundary.

One-dimensional CRFs

One of the simplest kinds of structures that are frequently found in structured prediction are one-dimensional sequences, which we can define the CRF²²

$$p(y_{1:T} | x) = \frac{1}{Z(x, \theta)} \prod_{t=1}^T \psi(y_t, x_t; \theta) \prod_{t=2}^T \psi(y_{t-1}, y_t; \theta) \quad (34)$$

One could consider a directed alternative model which factorizes based on the past history of conditional distributions $p(y_t | y_{t-1}, x_t; \theta)$.²³

This is called a *maximum entropy Markov model* (MEMM). However, relative to a CRF, there is a subtle difference where a MEMM is *locally normalized* for each conditional, while a CRF is *globally normalized* thanks to $Z(x, \theta)$. This allows information in a CRF to propagate through the entire sequence which helps enable more complex structures to be learned.

The most widely used area of application for one-dimensional CRFs historically has been natural language processing. Even now, when transformer based pure deep learning approaches dominate, there are many scenarios where we can combine methods to utilize the flexibility of purely data-driven approaches, with knowledge about the true constraints on the output space.

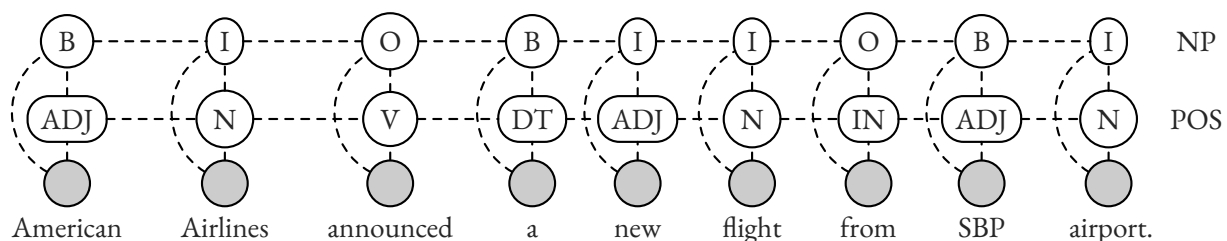


Figure 20: CRF for joint part of speech (POS) tagging and noun phrase (NP) segmentation. B: Begin NP, I: Within NP, O: Not in NP. N: Noun, ADJ: Adjective, V: Verb, IN: Preposition, DT: Determiner. After textbook Figure 4.30.

Textbook: §4.4, pp. 186–194

²¹Note how the partition function how depends on the inputs x as well as the parameters θ

Textbook: §4.4.1, pp. 187–190

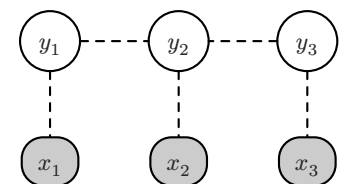


Figure 19: 1D CRF Example. After textbook Figure 4.29

²²This uses $\psi(y_t, x_t; \theta)$ to represent the *node potentials* which are the edges linking x and y at a time t , and uses $\psi(y_{t-1}, y_t; \theta)$ to represent the *edge potentials* which correspond to the edges linking y values at subsequent times. This factorization assumes the independence of the edge potentials from the actual values of x , but this assumption is strictly not required.

²³

Discussion

Consider how this is *different* than a Hidden Markov Model? Draw MEMM and HMM equivalents to the CRF example above to see.

Two-dimensional CRFs

As you might expect, in addition to one-dimensional sequence problems, the other most frequent use case for CRFs is in the other core pillar of applied ML domains, computer vision and thus two-dimensional image lattice data.²⁴

We can write out the conditional model in this case

$$p(y|x) = \frac{1}{Z(x)} \left[\sum_{i \neq j} \psi_{ij}(y_i, y_j) \right] \prod_i p(y_i | x_i) \quad (35)$$

Where we can either model a lattice of connecting neighboring nodes, or a *fully connected CRF* where every node is connected to every other node. This is often done when chaining models such as in image segmentation, where a convolutional neural network may take an input image and produce a score map at a coarse resolution of expected classes, which can then be interpolated to the full resolution and then passed through a fully connected CRF in order to increase the sharpness of the approximate segment boundaries.

Another interesting application of CRFs is in modeling *deformable parts*, that is in contexts such as object detection where instead of modeling the often too large space of complex objects, we can more easily model and detect more simple parts of the objects, and then add in constraints relating to the different ways those parts can be configured together.

Parameter estimation

In the general case a CRF can be written as follows

$$p(y | x; \theta) = \frac{\exp(f(x, y; \theta))}{Z(x; \theta)} = \frac{\exp(f(x, y; \theta))}{\sum_{y'} \exp(f(x, y'; \theta))} \quad (36)$$

Where $f(x, y; \theta)$ is a scoring or negative energy function, with high values corresponding to more probable configurations. We can then take the gradient of the log likelihood when optimizing to learn parameters

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} f(x_n, y_n; \theta) - \nabla_{\theta} \log(Z(x_n; \theta)) \quad (37)$$

As long as we can compute the corresponding expectations, this becomes tractable with the additional provision that since the partition function is dependent on the data value it is slower than in the MRF case.

Textbook: §4.4.2, pp. 190–193

²⁴ However, do not mistake this as being all that these models can be used for. There exist a wide range of structures beyond images and language both in the lattice style and other configurations for which a CRF may be a useful model to ensure structure to predictions.

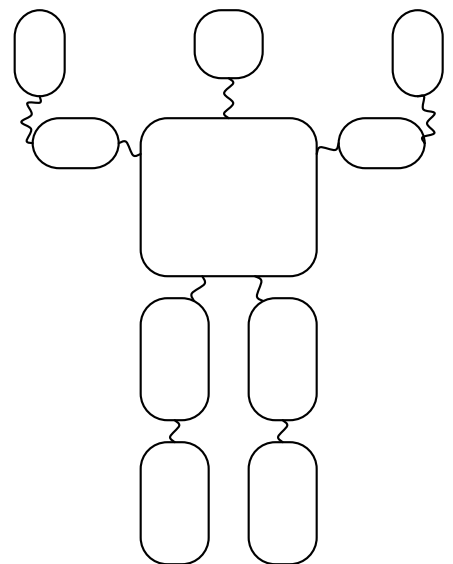


Figure 21: Visualization of deformable parts model for a human pose. Each part corresponds to a node in the CRF whose state space is the location of the part. The edges then represent pairwise spatial constraints (like springs), although note that because a CRF is a conditional model the “spring strength” can be input dependent. Local evidence nodes are omitted. After textbook Figure 4.35

Textbook: §§4.4.3–4.4.4, pp. 193–194

V Comparing directed and undirected PGMs

We have so far introduced two different languages for expressing probabilistic models. We have already started to see some scenarios where one of these languages may be more natural, but now let us try to formalize some of the differences.

Textbook: §4.5, pp. 194–202

Independence and factorization

The primary information encoded in any PGM is the set of CI assumptions asserted by the model. Thus we say a PGM is an **I-map** of a joint distribution p if the set of all independences asserted by the PGM $I(G)$ is a *subset* of the independences that are true of the model, that is $I(G) \subseteq I(p)$. However what is even stronger is if the graphical model can represent *all* (and only) the CI properties of the distribution. Such a PGM would be a **perfect map**, where $I(G) = I(p)$.

It turns out that DPGMs and UPGMs can be perfect maps to intersecting but non-identical sets of distributions. This means that, strictly speaking, neither is more powerful than the other as a representation language.

For example, consider a simple DPGM with only three nodes X, Y, Z in the form of a common effect triple. This model encodes the CI assumptions $\{X \perp\!\!\!\perp Y, X \not\perp\!\!\!\perp Y \mid Z\}$, which no MRF can capture, because undirected CI is monotonic. Similarly, a chordless cycle of length ≥ 4 has no DAG perfect map; any DAG I-map must add chords and thereby drop some independences.

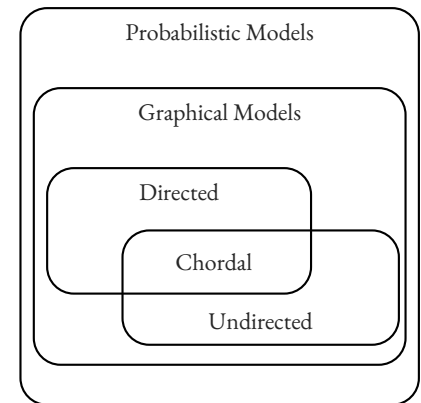


Figure 22: DPGMs and UPGMs can perfectly represent different sets of distributions. Some distributions can be perfectly represented by either DPGMs or UPGMs; the corresponding graph must be chordal.

After textbook Figure 4.36

Discussion

Show why the following PGMs cannot be translated between directed and undirected representations, both in the specific example attempts shown (produce a counter-example CI disagreement for each), as well as in general (sketch a proof). Finally, produce a graph that *can* be equivalently represented as both directed and undirected. What is special about this graph?

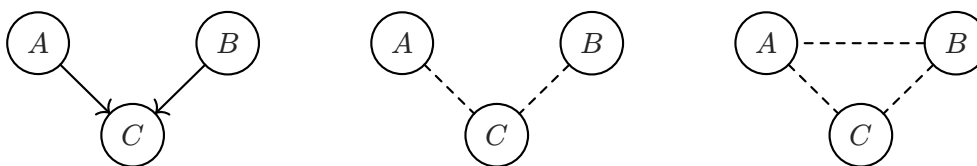


Figure 23: A common effect DPGM and MRFs which fail to represent the same independences.

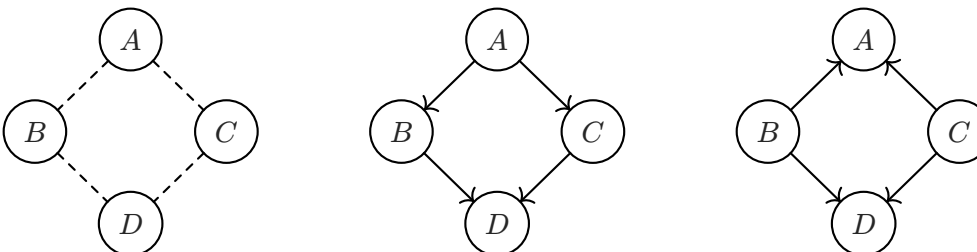


Figure 24: A MRF and DPGMs which fail to represent the same independences. After textbook Figure 4.37.

Chordal Models

However, despite the difference between languages in general, we have also seen that some distributions can be perfectly modeled by either a DPGM or UPGM. We call these distributions **chordal** or **decomposable**. Roughly speaking, this means the following: if we collapse together all the variables in each maximal clique, to make “mega-variables”, the resulting graph will be a tree. Of course, if the graph is already a tree (which includes chains as a special case), it will already be chordal. Another, equivalent but for us less useful, definition is that a chordal graph is one in which all cycles of four or more vertices have a chord, which is an edge that is not part of the cycle but connects two vertices of the cycle.

Translating PGMs

Even for non-chordal graphs, we may sometimes be ok with “lossy conversion” between representations where the resulting converted graph encodes *fewer* CI assumptions than the original. Such a conversion does not introduce anything false, it simply does not encode the full expressive power of the original. However, there still may be reasons to not mind this loss, and so we may wish to do such a conversion nonetheless.

Converting a DPGM to a UPGM

We have already seen the process of *moralization*, which takes a directed PGM and adds edges to connect all “unmarried parents”, and then drops the arrows. One nice property of moralized models are that all of the potentials are *locally normalized*, since they result from conditional distributions, which means that there is no need for a global normalization value ($Z = 1$).

Converting from UPGM to DPGM

We can also convert between an UPGM to a DPGM by means of the creation of *dummy nodes*. For each clique / potential $\psi_c(x_c; \theta_c)$, we create a new node, call it Y_c which is “clamped” to a special fixed value y_c^* . We then define the conditional distribution

$$p(Y_c = y_c^* \mid x_c) = \psi_c(x_c; \theta_c) \quad (38)$$

This “local evidence” CPD encodes the same factor as in the UPGM. This gives the overall joint of the form $p_{\text{undirected}}(x) \propto p_{\text{directed}}(x, y^*)$. By conditioning on the observed dummy state about which all nodes in a clique are parents, this forms repeated common effect triples which ensure the required dependencies.

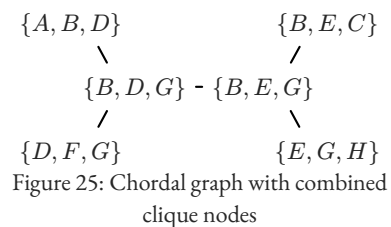
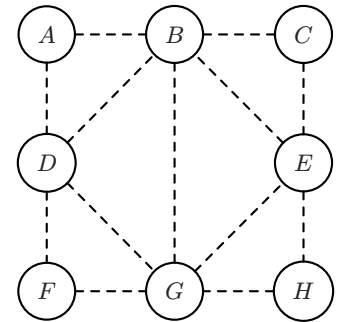


Figure 25: Chordal graph with combined clique nodes

Textbook: §4.5.2, pp. 196–197

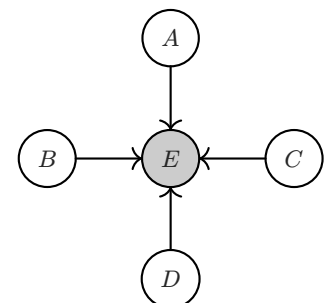
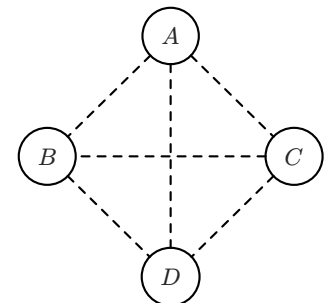


Figure 26: An undirected graphical model and directed equivalent using an observed dummy node. After textbook Figure 4.39

Discussion

Consider the following (conditional) undirected and directed graphical models.

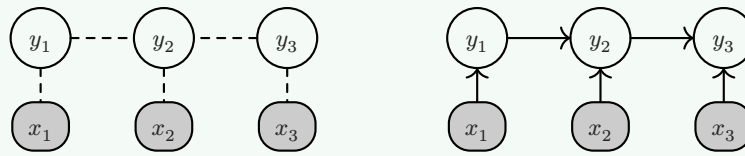


Figure 27: An undirected 1D CRF (left), and a directed MEMM (right). After textbook Figure 4.40

1. Momentarily, consider the MEMM graph without the conditional requirement. Draw the moralized conversion to a UPGM for this generalized MEMM. How does this differ from the CRF, both in graph structure and in terms of CI properties.
2. Consider the problem of using a CRF for structured prediction of grammatical sentences. What properties of the CRF, relative to the MEMM, would make this problem easier? *Hint: Consider the direction of dependence/information propagation required to determine if a noun is the subject or object of a sentence.*
3. On the other hand, what computational properties of the MEMM may make it more appropriate for modeling long sentences or online/streaming data?

Mixed Models

While some problems fit best using DPGMS and others UPGMs, in fact it is possible to define certain kinds of models that utilize *both* directed and undirected edges.

Textbook: §§4.5.4–4.5.5, pp. 198–202

Chain graphs

A **chain graph** (or partially directed acyclic graph, PDAG) is a PGM with both directed and undirected edges, *but without any directed cycles*. A Deep Belief Net is an example of a chain graph. Such a graph can be decomposed into a directed graph of *chain components*, which are maximal groups of nodes connected to each other only with undirected edges²⁵. We can then define a joint distribution

$$p(x) = \prod_i p(C_i \mid \text{parents}(C_i)) \quad (39)$$

25

Where each C_i is a chain component, and each CPD is a conditional random field.

Discussion

How is a chain component different than a maximal clique?

Discussion

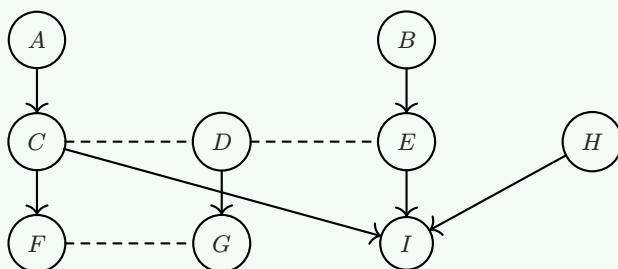


Figure 28: An example chain graph. After textbook Figure 4.42

For the above chain graph:

1. Identify all of the chain components
2. Sketch the reduced directed graph
3. Write out the full factorization of the joint

Acyclic Directed Mixed Graphs

A common problem that can occur in the analysis of DPGMs with respect to causal inference²⁶ is that different models with unobserved latent variables as common causes of observed variables can be hard or impossible to distinguish. One technique used to try and address some of this difficulty is the introduction of **acyclic directed mixed graphs (ADMG)**, which are graphs that have unidirectional edges as well as *bidirectional edges*²⁷ and no *directed cycles*.²⁸

Conditional independence can be determined for a ADMG using the criterion of *m-separation*, which is a generalization of d-separation that is equivalent to replacing each bidirectional edge $A \leftrightarrow B$ with a hidden variable such that $A \leftarrow H \rightarrow B$.

We can then define the **latent projection** of any DPGM to a ADMG with *no hidden variables* that is observationally equivalent to the original DPGM. The procedure for latent projection is as follows:

1. Initialize the projection ADMG with only nodes corresponding to the non-hidden variables of the DPGM.
2. Add an edge $A \rightarrow B$ if there is a directed path from A to B where every intermediate node is latent.
3. Add a bidirectional edge $A \leftrightarrow B$ if there exists a path between A and B such that all intermediate nodes are latent non-colliders (that is they do not have the common effect form $L_1 \rightarrow L_2 \leftarrow L_3$), and such that the edge adjacent to A and the edge adjacent to B both have arrowheads at those vertices. For example: $A \leftrightarrow \dots \rightarrow B$.

²⁶ A topic covered in more depth later in the course

²⁷ It is important to emphasize that these do different work than *undirected* edges with respect to joint factorization and CI properties.

²⁸ That is, there are no cycles in the graph composed entirely of directed edges, while bidirectional edges may create cycles

Discussion

Consider the following DPGM

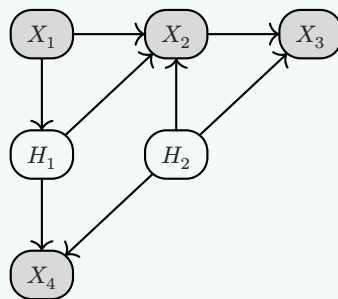


Figure 29: DPGM with latent variables. After textbook Figure 4.43.

1. Draw the latent reduction ADMG for the graph.
2. Verify that the set of CI properties of non-latent variables²⁹ in the original graph as determined through d-separation are identical to the CI properties of the latent reduction as verified through m-separation.

²⁹ Be careful about the distinction, which graphically is somewhat ambiguous, between observed variables in the model and conditioned variables in a CI query.

VI PGM extensions

In addition to the mixed methods discussed above, there are many formalizations that carve up joint distribution space using similar graphical languages to PGMs.

Textbook: §4.6, pp. 202–212

Factor graphs

A **factor graph** is a graphical representation that unifies directed and undirected models. They come in two common forms, bipartite factor graphs which have nodes for random variables and for factors, and Forney factor graphs which just have nodes for factors where edges represent random variables. Recall that factors are essentially the same as clique potentials, being functions whose product is unnormalized but proportional to the joint distribution.

Textbook: §4.6.1, pp. 202–205

Bipartite Factor Graphs

A bipartite factor graph is an undirected bipartite graph with two kinds of nodes. Round nodes represent random variables in the same way as standard PGMs. Square nodes (often also shaded or fully dark) represent factors with edges connecting each factor with all of the random variables included in it. It is easy to show how factor graphs form a super-set of UPGMs, but can model more distributions by introducing more factors. We can also represent any DPGM by introducing a factor for each CPD.

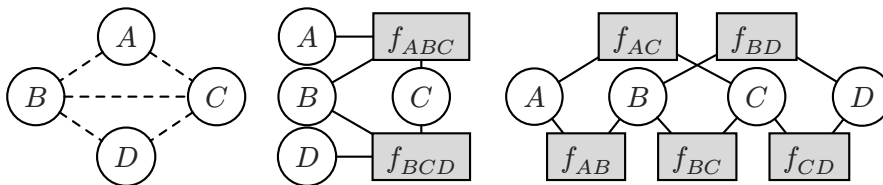


Figure 30: Simple UPGM and equivalent factor graphs. After textbook Figure 4.46.

Forney Factor Graphs

A **Forney factor graph (FFG)**, also sometimes called a normal factor graph, is a graph in which nodes represent factors and edges represent variables. This kind of visual language may be more familiar to those used to standard neural network diagrams or signal processing diagrams, where signals propagate along wires are modified by functions represented as nodes.

Note that in an FFG we may sometimes have a “half-edge” for variables that participate in only one factor. The directionality associated with the edges is a useful tool if there is a natural order in which the variables are generated, but strictly speaking the direction of the edges does not carry dependence information. In addition, associating directions with each edge allows us to uniquely name “messages” that are sent along each edge, which is useful for some kinds of inference algorithms. In addition to being more similar to neural network diagrams, FFGs have the advantage over bipartite FGs in that they support hierarchical (compositional) construction, in which a complex dependency structure between variables can be represented as a blackbox, with the input/output interface being represented by edges corresponding to the variables exposed by the blackbox.

Discussion

For the simple UPGM shown in the figure, draw as many other equivalent factor graphs as you can. Think about why in some applications some of these different models might be preferred over others.

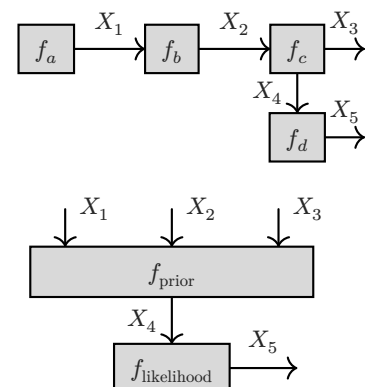


Figure 31: Example Forney factor graphs. A normal directed version on top, and a hierarchical abstracted version on bottom. After textbook Figure 4.48.

We can support variables participating in more than two factors in a FFG by adding *equality constraint nodes*. This is done by adding a factor defined using a Dirac/Kroneker delta function

$$f_{\text{eq}}(x, x_1, x_2) = \delta(x - x_1)\delta(x - x_2) \quad (40)$$

Intuitively we can think of this factor as acting like a “wire splitter”, producing two wires/edges/variables that have the same signal/value. Additionally we can sometimes re-use the same factor expression in multiple locations in the graph, an example of a technique called *parameter tying*.

Discussion

Parameter tying is a useful practice that can help sometimes reduce training and inference costs and encode certain kinds of dependencies. Consider the following two FFGs, one using parameter tying and the other simply connecting multiple ways to the same factor. Explain conceptually what is different between the two models, as well as how the factorizations of the joint differ.

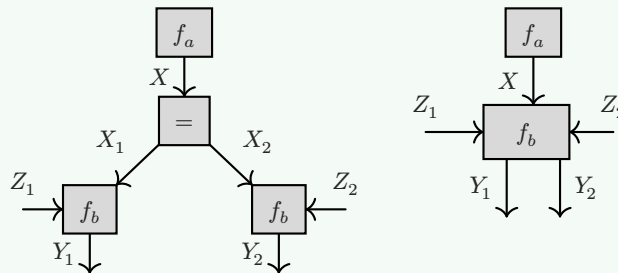


Figure 32: Example Fournery factor graphs. A normal directed version on top, and a hierarchical abstracted version on bottom. After textbook Figure 4.48.

Relational PGMs

We have seen how plate notation in DPGMs can help represent large repeating structures compactly, however not all repeating structures can be represented using plate notation (for instance a HMM). Many notational extensions to plate notation have been proposed but have not been widely adopted. These extensions are called **Relational Probability Models (RPM)**.

Textbook: §§4.6.3–4.6.6, pp. 206–210

Similar to first order logic, in an RPM we have constant symbols (representing objects), function symbols (mapping one set of constants to another), and predicate symbols (representing relations between objects). We will assume that each function has a type signature. Consider a network concerning online book reviews. We can define the predicates

$$\begin{aligned} \text{Honest}(H) &: \text{User} \rightarrow \{T, F\} \\ \text{Kindness}(K) &: \text{User} \rightarrow \{1, 2, 3, 4, 5\} \\ \text{Quality}(Q) &: \text{Book} \rightarrow \{1, 2, 3, 4, 5\} \\ \text{Rating}(R) &: \text{User} \times \text{Book} \rightarrow \{1, 2, 3, 4, 5\} \end{aligned} \quad (41)$$

Let us further state that there are two books B_1, B_2 and two users U_1, U_2 . The **basic random variables** in our model are obtained by instantiating each function with each possible combination of object inputs, (i.e. $H(U_2), R(U_1, B_2)$, etc) to create a set of **ground terms**.

In the directed case we will define our model in terms of a DPGM *template* which is defined in terms of generic index forms of the variables rather than their specific ground form. These can then be unrolled *as needed* to the actual variables depending on the specific inference query. Similarly, we could define our template in terms of a UPGM. These methods generally make use of parameter tying, as the parameters of the CPDs or potentials are generally constant in some way across all indexed variables.

Markov Logic Networks

One expressive way to represent URPMs is to use first order logic (FOL) rather than a standard graphical description. For example, consider a domain with the following rules: “All humans are mortal” and “If a mortal is friends with someone else, they are also mortal”³⁰. We can write this in FOL as

$$\begin{aligned} \forall H(x) &\Rightarrow M(x). \\ \forall F(x, y) \wedge M(x) &\Rightarrow M(y). \end{aligned} \quad (42)$$

Let us also say there are two objects (humans) in the world, Socrates (S) and Plato (P). We can then create the ground set of random variables such as $H(S), M(P), F(S, P)$, etc. Our goal is to define a joint probability distribution over these variables. We can do this by defining a UPGM with the ground variables and adding potential functions to capture each logical rule/constraint. For example to encode the rule $H(x) \Rightarrow M(x)$ we can define the potential and corresponding UPGM

$$\Psi(H(x), M(x)) = \begin{cases} 1 & \text{if } \neg H(x) \vee M(x) \\ 0 & \text{if } H(x) \wedge \neg M(x) \end{cases} \quad (43)$$

In comparison to purely logic based approaches, this method opens the door for us to be slightly less dogmatic and allow some probability for a rule to be violated. We can do this by changing the potentials from $\{0,1\}$ valued to exponentially weighted proportional to how hard we want the constraint to be³¹

$$\Psi(H(x), M(x)) = \begin{cases} e^w & \text{if } \neg H(x) \vee M(x) \\ e^0 & \text{if } H(x) \wedge \neg M(x) \end{cases} \quad (44)$$

While the full ground model is frequently prohibitively large to do inference, methods for **lifted inference** exists which only populate relevant components on the fly. Additionally there are methods to increase tractability by converting the discrete components of the model to continuous formulations that are easier to approximate and optimize which is the basis of **hinge-loss MRFs**. The template language for such models is known as **probabilistic soft logic**.

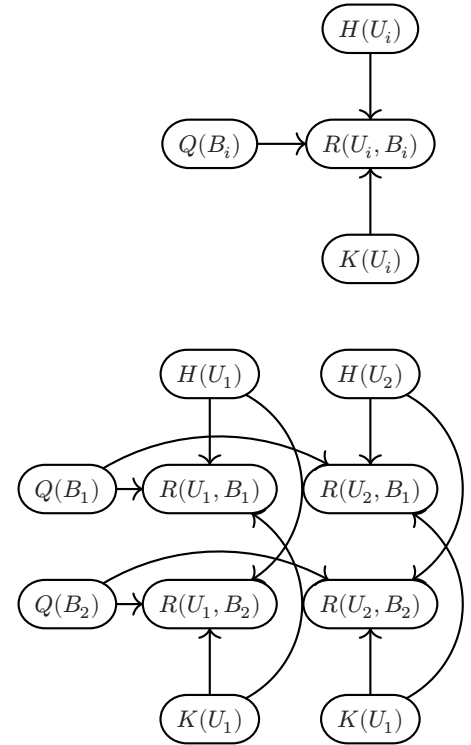


Figure 33: Template RPM and unrolled form for book review domain. After textbook Figure 4.50.

Textbook: §§4.6.4.2, pp. 209–211

³⁰ Immortal beings are notoriously unfriendly.

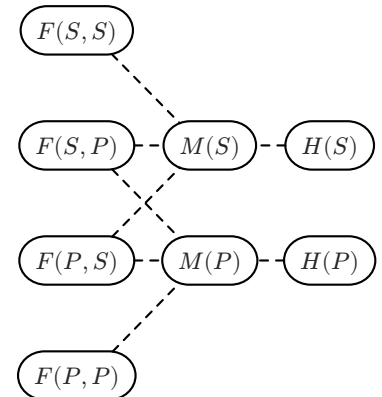


Figure 34: Markov logic network ground MRF example. After textbook Figure 4.52.

³¹ For instance if we want to leave the door open to immortal vampires being human.

Open universe Models

In all of the models we have discussed so far we have made at least one essential assumption as part of the problem statement. That is we are trying to represent some joint distribution for a set of random variables. Implicitly this makes what is called the **closed world assumption**, that is that the set of all objects is fixed and specified ahead of time. This is particularly unrealistic in the case of RPM where our actual specification of a template makes no actual statement about the number or specifics of objects. In many domains we want to represent relations between objects about which we do not know ahead of time how many there are. Another common context where such assumptions may not make sense is where we may have some *identification uncertainty*, that is we may have multiple measures but be uncertain whether these measures come from the same or different entities.

In such situations we can use **open-universe probability models (OUPM)** which can generate new objects as well as their properties. Early approaches to OUPM such as BLOG (Bayesian Logic) were general purpose but slow due to using MCMC over all possible worlds. Recently the library **Bean Machine**³² supports more efficient OUPMs.

Textbook: §§4.6.5, pp. 211

³²<https://beanmachine.org/>

Probabilistic Programming

The methods introduced so far have provided the *language* for communicating different kinds of probabilistic models. However, of course, these models do need to be computed. Traditionally all of the work to convert between the model specification and the relevant statistical computation was left manually to the developer (and sometimes this remains the case for particularly complex or niche analyses or models). However there exist libraries and domain-specific languages that better support direct representation of models and include to greater or less extents automatic inference or sampling. These are called **probabilistic programming languages (PPL)**, and they will be the main tools that you will likely use in developing your analysis for your final project. They can be more declarative or procedural, depending on their design or host language. Some good examples of PPLs you may want to look into are:

Textbook: §§4.6.6, pp. 211-212

- Pyro (Python, Particularly good Stochastic Variational Inference Support, <https://pyro.ai>)
- Bean Machine (Python, <https://beanmachine.org/>)
- PyMC (Python, <https://www.pymc.io/welcome.html>)
- Turing (Julia, <https://turinglang.org/index.html>)
- Gen (Julia, <https://www.gen.dev>)
- Fugue (Rust, <https://fugue.run/>)
- [And More](#)